

UNIVERSIDAD: Universidad Catolica San Pablo

CURSO: Sistemas Operativos CCOMP6-1

PROFESOR: Dr. Julio Santisteban

LABORATORIO N.º: 01

TEMA: Signaling y Saltos No Locales

DURACIÓN: 1 horas

1. Título del laboratorio

Laboratorio 01: Manejo de Señales (Signaling) y Tres Saltos No Locales con `sigsetjmp/siglongjmp` en C

2. Objetivo general

Comprender e implementar mecanismos de manejo de señales y control de flujo mediante saltos no locales en el lenguaje C dentro de un entorno Linux.

El estudiante deberá desarrollar un programa capaz de recibir tres tipos diferentes de señales, ejecutar un handler específico para cada señal y realizar un salto no local mediante `siglongjmp()` hacia un punto de recuperación previamente establecido mediante `sigsetjmp()`.

Después de procesar cada señal y realizar el salto correspondiente, el programa deberá continuar ejecutándose sin finalizar, permitiendo recibir nuevas señales.

Se deberán implementar:

- 3 señales diferentes.
- 3 signal handlers.
- 3 puntos de recuperación mediante `sigsetjmp()`.
- 3 saltos no locales mediante `siglongjmp()`.

3. Objetivos específicos

Al finalizar el laboratorio, el estudiante será capaz de:

1. Identificar el funcionamiento del sistema de señales en entornos Unix/Linux.
2. Registrar y configurar diferentes signal handlers utilizando `<signal.h>`.
3. Utilizar `sigsetjmp()` para establecer puntos de recuperación del flujo de ejecución.
4. Utilizar `siglongjmp()` para realizar saltos no locales desde un signal handler.
5. Diferenciar entre el flujo normal de ejecución y el flujo provocado por una señal.
6. Implementar tres mecanismos independientes de recuperación asociados a diferentes señales.
7. Enviar señales a un proceso mediante el comando `kill`.
8. Utilizar `Ctrl+C` para generar `SIGINT`.
9. Mantener el proceso activo después de recibir y procesar cada señal.

4. Recursos y herramientas necesarias

Sistema Operativo: GNU/Linux o entorno compatible con POSIX (WSL2, máquina virtual o instalación nativa).

Compilador: GCC.

Entorno de trabajo: Terminal de línea de comandos y editor de texto (VS Code, Vim, Nano, etc.).

Bibliotecas utilizadas:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <setjmp.h>
#include <unistd.h>
```

5. Introducción

Las señales son mecanismos utilizados por los sistemas Unix/Linux para notificar a un proceso sobre la ocurrencia de determinados eventos.

Una señal puede ser generada por el sistema operativo, por otro proceso o por el usuario. Por ejemplo:

- `SIGINT` puede generarse mediante `Ctrl+C`.
- `SIGUSR1` puede enviarse mediante `kill`.
- `SIGUSR2` puede enviarse mediante `kill`.

Normalmente, cuando un proceso recibe una señal, el sistema operativo interrumpe temporalmente su ejecución y ejecuta el `signal handler` asociado.

En este laboratorio se utilizará además el mecanismo de saltos no locales proporcionado por:

```
sigsetjmp()
siglongjmp()
```

`sigsetjmp()` permite establecer un punto de recuperación dentro del flujo de ejecución del programa. Posteriormente, un `signal handler` puede utilizar `siglongjmp()` para transferir el control directamente hacia dicho punto.

El laboratorio implementará tres mecanismos independientes de recuperación:

```
SIGINT  → Handler 1 → siglongjmp() → Punto 1
SIGUSR1 → Handler 2 → siglongjmp() → Punto 2
SIGUSR2 → Handler 3 → siglongjmp() → Punto 3
```

Después de cada salto, el programa deberá continuar ejecutándose y permanecer disponible para recibir nuevas señales.

6. Señales utilizadas

Señal	Forma de generarla	Handler	Punto de recuperación
<code>SIGINT</code>	<code>Ctrl+C</code>	<code>handler_sigint()</code>	<code>salto_sigint</code>
<code>SIGUSR1</code>	<code>kill -SIGUSR1</code>	<code>handler_sigusr1(</code>	<code>salto_sigusr1</code>
<code>1</code>	<code>PID</code>	<code>)</code>	

```
SIGUSR kill -SIGUSR2      handler_sigusr2( salto_sigusr2
2      PID                )
```

De esta manera, el estudiante deberá implementar tres saltos no locales.

7. Instrucciones del laboratorio

Paso 1 — Preparar el entorno

Abrir dos terminales:

Terminal 1: ejecución del programa.

Terminal 2: envío de señales mediante `kill`.

Crear el directorio de trabajo:

```
mkdir laboratorio01
cd laboratorio01
```

Paso 2 — Crear el programa

Crear el archivo:

```
ejercicio1.c
```

El programa deberá contener tres buffers de contexto:

```
sigjmp_buf salto_sigint;
sigjmp_buf salto_sigusr1;
sigjmp_buf salto_sigusr2;
```

Cada buffer representará un punto de recuperación diferente.

8. Ejercicio práctico

Ejercicio 1 — Tres señales y tres saltos no locales

Implementar un programa que registre los siguientes handlers:

```
void handler_sigint(int signal);
void handler_sigusr1(int signal);
void handler_sigusr2(int signal);
```

Los handlers deberán estar asociados respectivamente a:

```
SIGINT
SIGUSR1
SIGUSR2
```

Handler de SIGINT

Cuando el usuario presione:

```
Ctrl+C
```

se deberá ejecutar:

```
siglongjmp(salto_sigint, 1);
```

El programa deberá regresar al punto establecido por:

```
sigsetjmp(salto_sigint, 1);
```

y mostrar un mensaje indicando que se produjo el salto.

Handler de SIGUSR1

Desde la segunda terminal se deberá ejecutar:

```
kill -SIGUSR1 <PID>
```

El handler deberá realizar:

```
siglongjmp(salto_sigusr1, 1);
```

El programa deberá regresar al punto establecido mediante:

```
sigsetjmp(salto_sigusr1, 1);
```

y mostrar un mensaje indicando que se produjo el segundo salto.

Handler de SIGUSR2

Desde la segunda terminal se deberá ejecutar:

```
kill -SIGUSR2 <PID>
```

El handler deberá realizar:

```
siglongjmp(salto_sigusr2, 1);
```

El programa deberá regresar al punto establecido mediante:

```
sigsetjmp(salto_sigusr2, 1);
```

y mostrar un mensaje indicando que se produjo el tercer salto.

9. Comportamiento esperado

El programa deberá permanecer ejecutándose dentro de un ciclo.

Por ejemplo:

```
=====
LABORATORIO 01
SIGNALING Y SALTOS NO LOCALES
=====
```

PID del proceso: 12345

Programa en ejecución...

Esperando señales...

Si el estudiante presiona:

Ctrl+C

deberá observar algo similar a:

```
[ SIGINT recibida ]
```

```
>>> Ejecutando salto no local #1
```

```
>>> Recuperación en punto SIGINT
```

Programa continúa ejecutándose...

Esperando señales...

Posteriormente, desde la Terminal 2:

```
kill -SIGUSR1 12345
```

deberá aparecer:

```
[ SIGUSR1 recibida ]
```

```
>>> Ejecutando salto no local #2
```

```
>>> Recuperación en punto SIGUSR1
```

Programa continúa ejecutándose...

Esperando señales...

Finalmente:

```
kill -SIGUSR2 12345
```

deberá producir:

```
[ SIGUSR2 recibida ]
```

```
>>> Ejecutando salto no local #3
```

```
>>> Recuperación en punto SIGUSR2
```

Programa continúa ejecutándose...

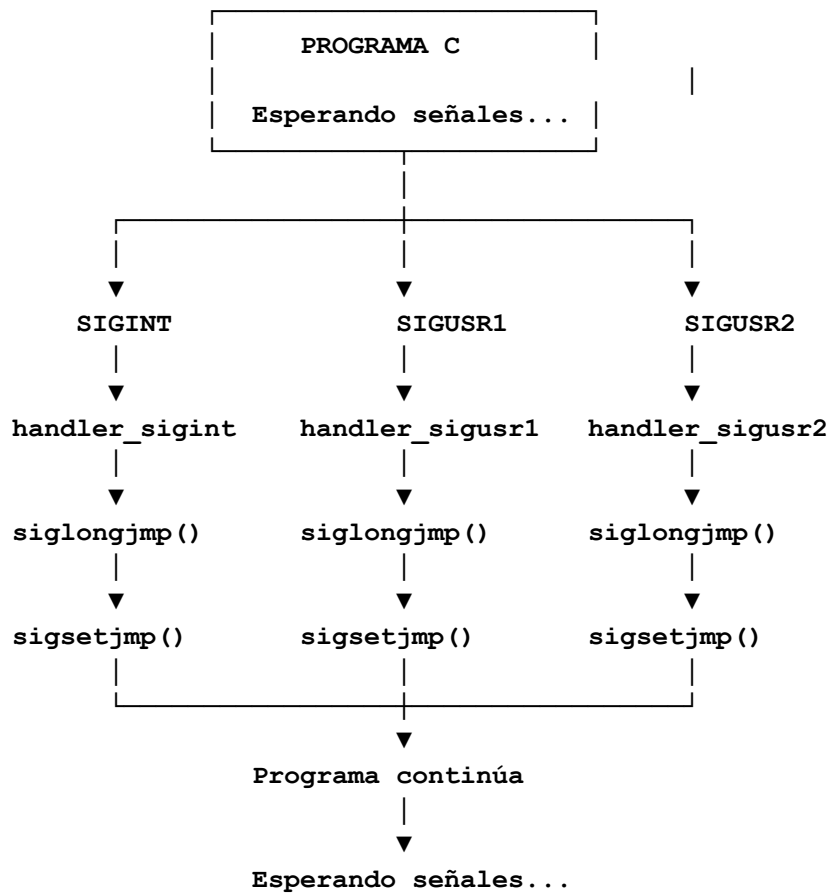
Esperando señales...

El proceso no debe finalizar después de ninguno de los tres saltos.

El estudiante deberá poder repetir las señales varias veces.

10. Requisito fundamental

El programa debe cumplir el siguiente comportamiento:



El proceso no debe terminar después de recibir ninguna de las tres señales.

11. Compilación

Compilar utilizando:

```
gcc -Wall -Wextra ejercicio1.c -o ejercicio1
```

El programa deberá compilar sin errores y, preferentemente, sin advertencias.

Ejecutar:

```
./ejercicio1
```

Obtener el PID utilizando, por ejemplo:

```
ps
```

o directamente desde el programa mediante:

```
printf("PID: %d\n", getpid());
```

12. Pruebas

Desde la segunda terminal realizar las siguientes pruebas.

Prueba 1 — SIGINT

Presionar:

Ctrl+C

Verificar:

- recepción de SIGINT;
- ejecución del handler;
- ejecución de `siglongjmp()`;
- recuperación mediante `sigsetjmp()`;
- continuidad del programa.

Prueba 2 — SIGUSR1

`kill -SIGUSR1 <PID>`

Verificar:

- recepción de SIGUSR1;
- ejecución del segundo handler;
- segundo `siglongjmp()`;
- recuperación del programa;
- continuidad del proceso.

Prueba 3 — SIGUSR2

`kill -SIGUSR2 <PID>`

Verificar:

- recepción de SIGUSR2;
- ejecución del tercer handler;
- tercer `siglongjmp()`;
- recuperación del programa;
- continuidad del proceso.

13. Criterios de evaluación

Criterio	Ponderación	Descripción
Implementación de SIGINT	20%	Registro correcto del handler, uso de <code>sigsetjmp/siglongjmp</code> y recuperación del flujo.
Implementación de SIGUSR1	20%	Registro correcto del handler, recepción de la señal y salto no local.
Implementación de SIGUSR2	20%	Registro correcto del handler, recepción de la señal y salto no local.
Continuidad del proceso	20%	El programa permanece ejecutándose después de cada uno de los tres saltos.
Uso de herramientas CLI	10%	Identificación del PID y envío correcto de señales mediante <code>kill</code> y <code>Ctrl+C</code> .
Calidad de código	10%	Código estructurado, uso adecuado de bibliotecas y compilación con <code>-Wall -Wextra</code> .

Total: 100%

14. Entregable

El estudiante deberá entregar:

`ejercicio1.c`

El programa debe implementar:

- `SIGINT`
- `SIGUSR1`
- `SIGUSR2`
- 3 `signal` handlers
- 3 `sigsetjmp()`
- 3 `siglongjmp()`
- continuidad del proceso después de cada salto.